Explicit Type Conversion (Type Casting) in JavaScript

Explicit conversion means YOU manually convert one type into another.

Main conversions in JavaScript:

1. String Conversion
2. Number Conversion
3. Boolean Conversion
4. BigInt Conversion
5. Object → Primitive Conversion

---

1. String Conversion

Ways to convert into string:

String(value)
value.toString()
value + ""
`${value}`

Using `String()`

String(123)

Result:

"123"

String(true)

Result:

"true"

String(null)

Result:

"null"

Using `.toString()`

(123).toString()

Result:

"123"

But:

null.toString()

❌ Error

Because:

null and undefined don't have methods

Using `+ ""`

123 + ""

Result:

"123"

Because + with string triggers string coercion.

---

2. Number Conversion

Ways:

Number(value)
parseInt(value)
parseFloat(value)
+value

Using `Number()`

Number("123")

Result:

123

Number(true)

Result:

1

Number(false)

Result:

0

Number(null)

Result:

0

Number(undefined)

Result:

NaN

Number("abc")

Result:

NaN

Unary Plus (`+value`)

+"123"

Result:

123

+true

Result:

1

---

parseInt()

Reads integer until invalid character.

parseInt("123px")

Result:

123

---

parseInt("12.9")

Result:

12

---

parseFloat()

parseFloat("12.9px")

Result:

12.9

---

3. Boolean Conversion

Ways:

Boolean(value)
!!value

Falsy Values

Only these become false:

false

0
-0
0n
""
null
undefined
NaN

Everything else is true.

Examples

Boolean(1)

Result:

true

Boolean("")

Result:

false

Double NOT (!!)

!!"hello"

Result:

true

!!0

Result:

false

---

4. BigInt Conversion

BigInt(10)

Result:

10n

BigInt("123")

Result:

123n

Cannot use decimal:

BigInt(1.5)

❌ Error

---

5. Object to Primitive Conversion

Objects first convert to primitive.

JS tries:

valueOf()
toString()

Example

```
const obj = {
  valueOf() {
    return 10;
  }
};

Number(obj);
```

Result:

10

How Operators Work

Now the important part.

+ Operator

+ is special.

It can do:

1. Addition
2. String concatenation

Rule of +

If one operand becomes string → concatenation.

Otherwise → numeric addition.

Example 1

"5" + 2

Step:

2 → "2"

Result:

"52"

Example 2

5 + true

Step:

true → 1

Result:

6

Example 3

5 + null

Step:

null → 0

Result:

5

Example 4

5 + undefined

Step:

undefined → NaN

Result:

NaN

-, *, /

These ALWAYS try numeric conversion.

Example

"10" - 2

Step:

"10" → 10

Result:

8

"10" * "2"

Result:

20

"20" / "5"

Result:

4

Comparison Operators

"5" > 2

Step:

"5" → 5

Result:

true

But string vs string is lexicographical.

"20" > "3"

Result:

false

Because:

"2" < "3"

character comparison.